

DOCKER

COMPLETE GUIDE TO DOCKER FOR BEGINNERS
AND INTERMEDIATES

Questions and Answers



DAY 2

MASTER DEVOPS WITH VINAY

11: What are the main components of Docker architecture?

Answer:

Docker's architecture is built around several core components that work together to enable containerized application development and deployment:

1. Docker Engine

The heart of Docker, responsible for creating and managing containers. It includes:

- Docker Daemon (dockerd): A background service that manages Docker objects like containers, images, networks, and volumes.
- Docker CLI (docker): A command-line interface to interact with the Docker Daemon.
- REST API: Allows external programs and tools to communicate with the Docker Daemon.

2. Docker Images

Read-only, reusable templates that define the contents and configuration of containers, including the application code, runtime, libraries, and dependencies.

3. Docker Containers

Lightweight, standalone, and executable instances of Docker images. They package the application and its environment, ensuring consistency across different environments.

4. Docker Hub

A cloud-based registry where Docker users can store, share, and download images publicly or privately.

5. Docker Volumes

A feature for persisting and managing data generated by and used within containers, independent of the container lifecycle.

12. What is Docker Hub, and how do developers use it?

Answer:

Docker Hub is Docker's official cloud-based registry service for storing, managing, and sharing container images. It acts as a central repository where users can access both public and private Docker images.

Key Features:

1. Public Repositories:

Access a wide range of pre-built images for popular applications like nginx, mysql, ubuntu, and more.

2. Private Repositories:

Securely store proprietary or internal-use images with controlled access.

3. Automated Builds:

Automatically build images from source code hosted on repositories like GitHub or Bitbucket.

4. Webhooks:

Integrate with CI/CD pipelines by triggering actions whenever an image is updated.

Common Usage Examples:

- Pull an image:
`docker pull nginx:latest`
- Push an image:
`docker push <username>/<repository-name>`

13. Can you describe the role and responsibilities of the Docker Daemon in Docker's architecture?

Answer:

The Docker Daemon (dockerd) is the core background service in Docker's architecture. It is responsible for managing all Docker-related components, including containers, images, networks, and volumes.

Key Responsibilities:

1. Container Lifecycle Management:

Handles the creation, starting, stopping, and removal of containers.

2. Image Management:

Builds Docker images and manages pulling from or pushing to registries like Docker Hub.

3. Networking:

Configures and manages networking between containers and external systems.

4. API Server:

Listens for commands from the Docker CLI or through the REST API, enabling remote and automated interaction with Docker.

14. What is a Dockerfile, and what makes it essential in Docker workflows?

Answer:

A Dockerfile is a simple text file that contains a set of instructions used to build a Docker image. It defines the environment, dependencies, and commands needed to package an application into a container, enabling automation, consistency, and repeatability in image creation.

Common Instructions in a Dockerfile:

1. FROM

Specifies the base image for the build.

Example:

```
FROM ubuntu:20.04
```

2. RUN

Executes commands during the image build process.

Example:

```
RUN apt-get update && apt-get install -y nginx
```

3. COPY

Copies files or directories from the host into the image.

Example:

```
COPY index.html /var/www/html/
```

4. CMD

Sets the default command to run when the container starts.

Example:

```
CMD ["nginx", "-g", "daemon off;"]
```

5. EXPOSE

Declares which ports the container will listen on at runtime.

Example:

```
EXPOSE 80
```

Why It's Important:

- Ensures automated and consistent image builds across environments.
- Facilitates easy sharing, versioning, and reproducibility of application setups.

15. Can you explain the steps involved in building a Docker image using a Dockerfile?

Answer:

To build a Docker image from a Dockerfile, follow these steps:

Steps to Build an Image:

1. Create a Dockerfile containing all the necessary build instructions.
2. Run the docker build command to create the image.
3. Assign a name and optional version tag to the image during the build.

Command Syntax:

```
docker build -t <image-name>:<tag> <path-to-dockerfile>
```

Example:

```
docker build -t my-app:1.0 .
```

- -t : Tags the image with a name and version (my-app:1.0).
- . : Refers to the current directory where the Dockerfile is located.

Explanation:

- The Docker Daemon reads the Dockerfile line by line and executes each instruction to build the image.
- Each instruction creates a new layer, enabling efficient caching and reuse in future builds.

16. What is the difference between docker run and docker start commands?

Answer:

Both commands are used to start containers, but they serve different purposes in the container lifecycle.

Command	Description
docker run	Creates a new container from a specified image and then starts it.
docker start	Starts an existing, stopped container without creating a new one.

Key Differences:

docker run is used when launching a container for the first time from an image.

docker start is used to restart a previously created (but stopped) container.

Examples:

```
docker run -d -p 8080:80 nginx
```

Creates and starts a new container from the nginx image

```
docker start <container-id>
```

Starts an existing container that was previously stopped

17. How can you view all Docker images and containers on a system?

Answer:

Docker provides simple commands to view both images and containers on your system.

To List Docker Images:

```
docker images
```

Output Includes:

- Repository name
- Tag
- Image ID
- Created time
- Size

This command shows all images stored locally on your machine.

To List Docker Containers:

1. Running Containers Only:

```
docker ps
```

2. All Containers (Running + Stopped):

```
docker ps -a
```

Output Includes:

- Container ID
- Name
- Status (e.g., running, exited)
- Image used
- Port mappings
- Creation time

Explanation:

- docker images lists all available Docker images locally.
- docker ps and docker ps -a help track the status and history of your containers.

18. What is the command to delete a Docker image?

Answer:

To delete a Docker image, use the `docker rmi` command.

Steps to Delete an Image:

1. List available images:
`docker images`
2. Remove an image by name or ID:
`docker rmi <image-id>`
or
`docker rmi <image-name>`
3. Force removal (if the image is used by a container):
`docker rmi -f <image-id>`

Explanation:

- Docker will not delete an image if it's associated with any running or stopped container unless you use the `-f` (force) flag.
- Always ensure the image is no longer in use before removal to avoid unintended impact.

19. How do you inspect a Docker container's logs?

Answer:

To view the logs of a running or stopped Docker container, use the `docker logs` command.

Basic Command:

`docker logs <container-id>`

Common Options:

1. Real-Time (Live) Logs:
`docker logs -f <container-id>`
- `-f`: Streams the log output in real time (follow mode).
2. View Recent Log Lines:
`docker logs --tail 10 <container-id>`
- `--tail 10`: Shows only the last 10 lines of the log output.

These options are especially useful for debugging or monitoring container behavior without attaching directly to the container process. Let me know if you need examples for structured logging or multi-container setups!

20. What is a Docker volume?

Answer:

A Docker volume is a persistent storage mechanism used to retain data generated or used by containers, even after the container is stopped or removed. Volumes are managed by Docker and are separate from the host's filesystem structure.

Key Features:

1. Persistent Storage: Data stored in volumes remains intact even if the container is deleted.
2. Container Sharing: Volumes can be mounted to and shared across multiple containers.
3. Docker-Managed: Volumes are handled using Docker CLI commands and are isolated from the container's image or the host system.

Example: Creating and Mounting a Volume

`docker volume create my-volume`

`docker run -v my-volume:/data alpine`

- `my-volume` is the name of the volume.
- `/data` is the path inside the container where the volume is mounted.